

Aaruush Data Vortex — Round 2

Technical Report — Rebuilding the Semantic Comprehension Layer

1. Problem Definition

The Social Engine's semantic comprehension layer has failed and can no longer interpret tone or intent behind user text. Dataset 2 was recovered from a deliberately obfuscated PDF: each field (text_id, post_text, sentiment_label, topic_category) was rendered as a separate overlapping text object at a fixed column position, so any naive left-to-right text extraction interleaves the characters into garbage. A custom extractor (extract_dataset.py) re-separates the columns using their true fixed x-coordinates rather than reading order, recovering 9,000 clean, labeled rows with zero nulls and zero duplicates. The task selected is 3-class sentiment classification (Positive / Negative / Neutral, perfectly balanced at 3,000 each), the most directly useful semantic signal for a social platform to recover first, and the same output format needed for any downstream moderation or ranking logic.

2. Preprocessing Pipeline

Applied to post_text before feature extraction, in order:

1. Lowercase
2. Strip @user mentions (anonymized placeholder, carries no sentiment)
3. Strip URLs
4. Strip '#' from hashtags but keep the word itself (e.g. #HLWD -> hlwd)
5. Fix stray unicode escape artifacts (\u2019 -> apostrophe)
6. Strip standalone numbers
7. Strip punctuation
8. Tokenize on whitespace, drop stopwords EXCEPT negation words (no/not/Isn't/wasn't/can't...) since stripping negation flips meaning
9. Drop tokens of length <= 1

All 9,000 rows retained non-empty text after cleaning. Features: TF-IDF, unigrams + bigrams (bigrams needed to catch negated phrases like "not good" that unigrams alone would treat as just "good"), max 8,000 features, min document frequency 3, sublinear TF scaling.

3. Model Selection & Justification

Three standard text-classification baselines were trained and compared on identical TF-IDF features: Multinomial Naive Bayes, Logistic Regression, and Linear SVM. All three are appropriate for high-dimensional sparse text features and fast to train/evaluate, allowing a fair head-to-head comparison rather than committing to one architecture upfront.

Model	Accuracy	Macro F1
Multinomial Naive Bayes	0.6128	0.6114
Logistic Regression (selected)	0.6406	0.6416
Linear SVM	0.6311	0.6319

Logistic Regression was selected: best macro-F1 (0.642). Macro-F1 rather than accuracy drove this decision since accuracy alone can hide a model that's strong on two easy classes and weak on the hard one, macro-F1 weights all three classes equally regardless of how easy they are.

4. Training Methodology

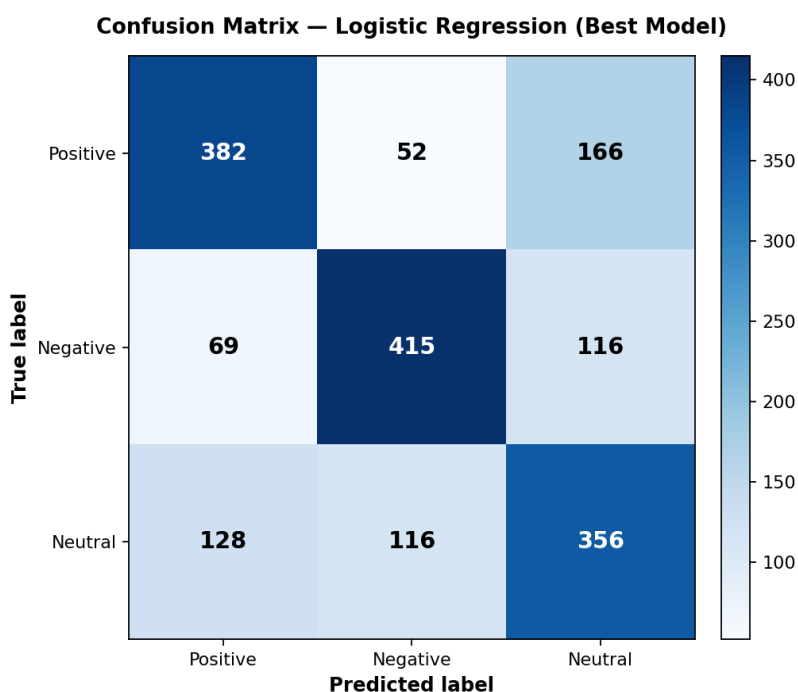
80/20 stratified train/test split (7,200 train, 1,800 test), stratified on sentiment_label to preserve the 1:1:1 class balance in both splits. Random state fixed (42) for reproducibility. Logistic Regression trained with L2 regularization ($C=1.0$), max_iter=1,000, on the TF-IDF matrix described above. No separate validation set was carved out beyond the test split, given the dataset size (9,000 rows) and the goal of a fast, reproducible baseline rather than extensive hyperparameter search; $C=1.0$ is scikit-learn's default and was not tuned further.

5. Evaluation Metrics

Class	Precision	Recall	F1-score	Support
Negative	0.712	0.692	0.702	600
Neutral	0.558	0.593	0.575	600
Positive	0.660	0.637	0.648	600
Macro avg	0.643	0.641	0.642	1800

Overall test accuracy: 64.1% (1,800 test posts).

6. Confusion Matrix



7. Error Analysis

Of 1,800 test predictions, 647 (35.9%) were wrong. Errors concentrate almost entirely around Neutral: 128 true-Neutral posts predicted Positive, 116 predicted Negative, and conversely 166 true-Positive and 116 true-Negative posts were predicted Neutral.

Negative-vs-Positive confusion is comparatively rare (52 and 69 cases), meaning the model can usually tell clear positive and negative tone apart, its actual weakness is separating mild/ambiguous sentiment from genuinely neutral statements.

Representative misclassified examples:

Neutral -> Positive: "burger blonde friday today crows nest pure blonde burger purchase"

Neutral -> Negative: "st writer implies copied kpop lectures know whats korean word backtrack"

Positive -> Neutral: "john cena throwing jon stewart wwe pretty amusing"

Negative -> Neutral: "st time drove stick ever kinda lied salesman ab honda drive standard"

Common thread: these posts lack explicit sentiment vocabulary

("good"/"bad"/"love"/"hate"). Sentiment is implicit, carried by real-world context (a WWE moment being "amusing", a lied-to car salesman story) that a bag-of-words TF-IDF model has no way to access, it can only react to the specific words present, not world knowledge about what those words imply. Two concrete gaps: (1) short, list-like posts (event/venue names, e.g. the burger example) carry near-zero sentiment-bearing vocabulary at all, making Neutral vs Positive genuinely ambiguous even for a human without more context; (2) sarcasm and understatement ("pretty amusing", "kinda lied") use mild-intensity words that TF-IDF weights the same regardless of the surrounding narrative tone. A

transformer-based model (e.g. a fine-tuned DistilBERT) would likely close part of this gap by modeling word order and context rather than treating text as an unordered bag of terms, at the cost of significantly more training time and compute than this TF-IDF baseline needed.